

Relatório Prático: Primeiro Container com Docker

Disciplina: Computação em Nuvem

Integrante: Wesley Azevedo Melo

1. Introdução

Este relatório documenta a execução e os resultados obtidos no Exercício 4 da disciplina de Computação em Nuvem. O objetivo prático da atividade foi compreender a arquitetura e o funcionamento do ecossistema de containers utilizando a plataforma Docker.

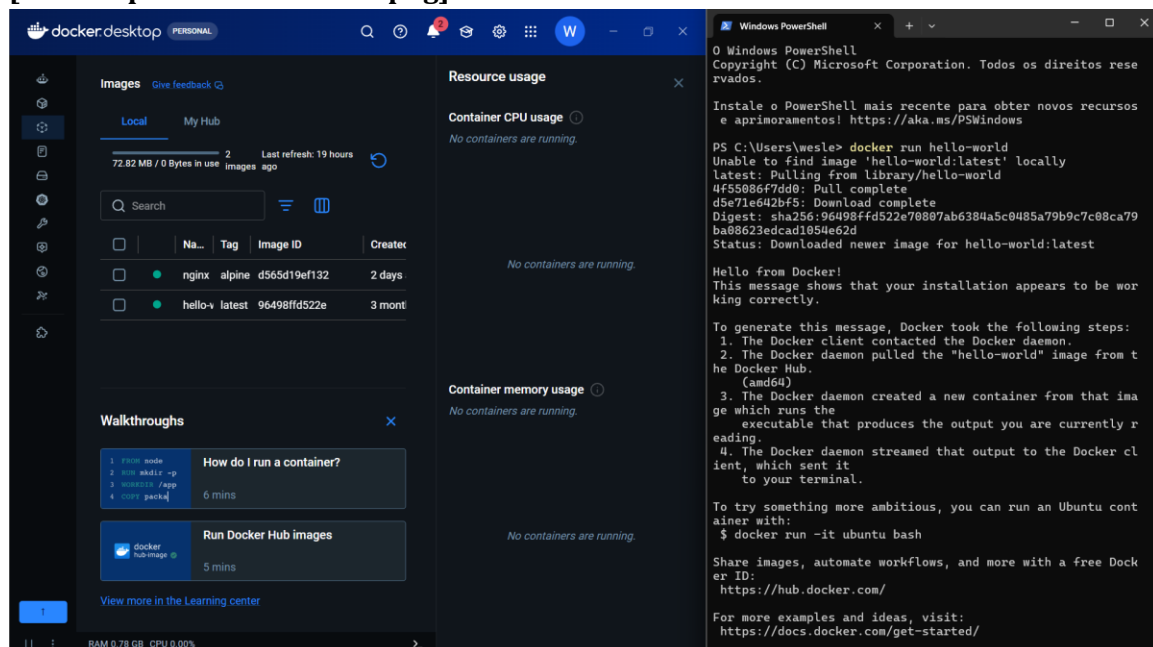
A atividade envolveu desde a validação inicial do ambiente de execução com uma imagem de teste padronizada (hello-world) até o desenvolvimento completo de uma aplicação personalizada em linguagem Python. Através da criação de um arquivo de configuração Dockerfile, foi possível realizar o empacotamento (build) e a execução (run) da aplicação de forma totalmente isolada e independente do sistema operacional hospedeiro.

2. Evidências Práticas (Capturas de Tela)

Abaixo as capturas de tela coletadas durante o processo:

2.1 Rodando o Hello World

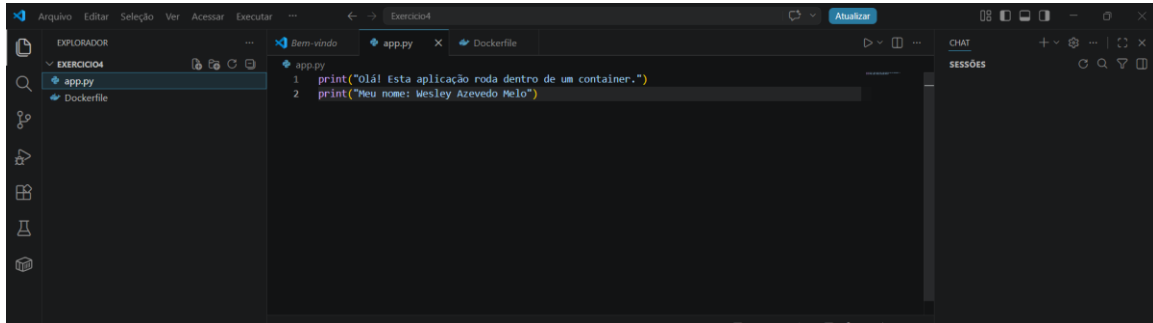
[Print 1: print1-hello-world.png]



Evidência do comando `docker run hello-world` sendo executado com sucesso no terminal, exibindo a mensagem de boas-vindas do Docker.

2.2 Criação do Arquivo de Aplicação (app.py)

[Print 2: print2-app-py.png]

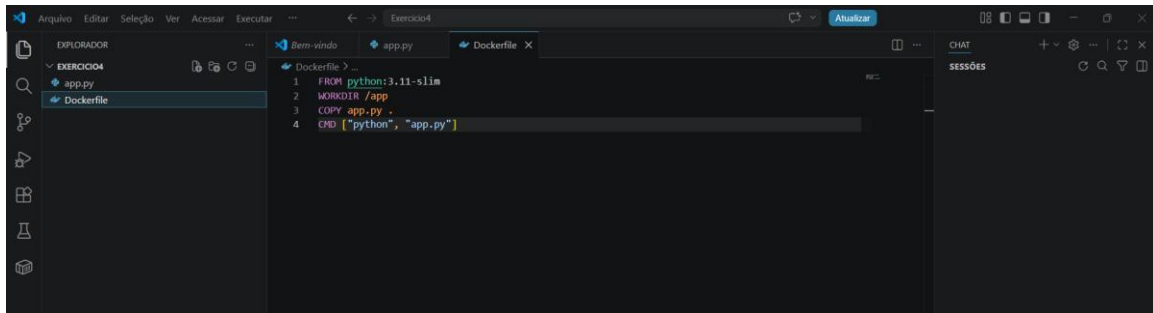
A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows a project named 'EXERCICIO4' with two files: 'app.py' and 'Dockerfile'. The 'app.py' file is selected and its content is displayed in the main editor area. The code consists of two lines: a print statement with a greeting and a print statement with a name.

```
1 print("Olá! Esta aplicação roda dentro de um container.")
2 print("Meu nome: Wesley Azevedo Melo")
```

Evidência do arquivo `app.py` criado no VS Code contendo as instruções de print e o nome completo do aluno.

2.3 Criação das Instruções de Build (Dockerfile)

[Print 3: print3-dockerfile.png]

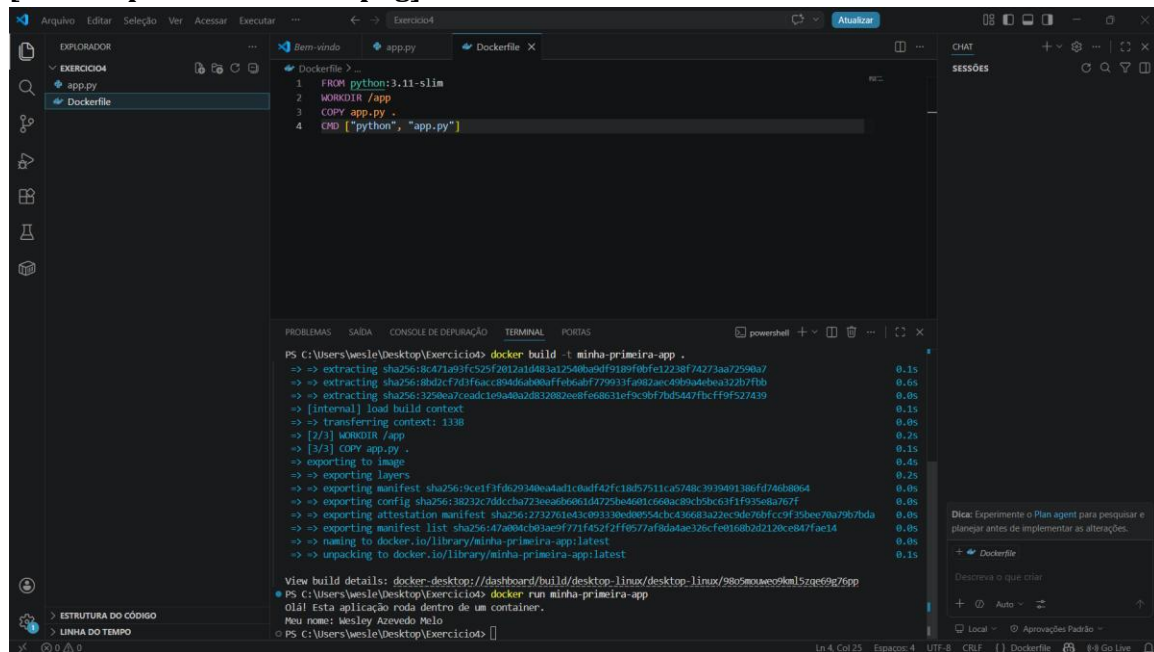
A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows the same project 'EXERCICIO4' with 'app.py' and 'Dockerfile'. The 'Dockerfile' file is selected and its content is displayed in the main editor area. The Dockerfile contains four instructions: setting the base image to Python 3.11-slim, setting the working directory to /app, copying the app.py file, and running the application with python.

```
1 FROM python:3.11-slim
2 WORKDIR /app
3 COPY app.py .
4 CMD ["python", "app.py"]
```

Evidência do arquivo `Dockerfile` configurado com a imagem base do Python, definição do diretório de trabalho e comandos de inicialização.

2.4 Construção e Execução da Imagem Customizada

[Print 4: print4-build-run.png]



The screenshot shows a code editor with a Dockerfile in the editor and its build output in the terminal. The Dockerfile contains the following instructions:

```
1 FROM python:3.11-slim
2 WORKDIR /app
3 COPY app.py .
4 CMD ["python", "app.py"]
```

The terminal output shows the build process for the image named 'minha-primeira-app'. The build steps include extracting the base image, copying the application code, and exporting the image. The final output is the image name 'minha-primeira-app'.

Evidência do terminal exibindo o processo de compilação da imagem com `docker build` e a saída final com seu nome impressa através do comando `docker run`.

3. Reflexão e Respostas Teóricas

Qual a diferença entre uma imagem e um container?

Imagem: É um pacote estático, imutável e de leitura que contém todo o código-fonte, bibliotecas de sistema, dependências e configurações necessárias para que uma aplicação funcione. Pode ser compreendida como o "molde", "blueprint" ou o instalador de um software.

Container: É a instância ativa, viva e em execução de uma imagem. Ele representa o processo real isolado na memória do computador. A partir de um único molde (imagem), é possível instanciar e rodar múltiplos ambientes idênticos (containers) em paralelo.

Por que o container é mais leve que uma VM (Máquina Virtual)?

As Máquinas Virtuais (VMs) necessitam virtualizar toda a infraestrutura de hardware físico por meio de um Hypervisor. Cada VM obrigatoriamente carrega consigo um Sistema Operacional (OS) convidado completo e redundante, o que consome gigabytes de espaço em disco e exige minutos para inicializar.

Os containers, por outro lado, realizam a virtualização a nível de software (espaço de usuário). Eles não possuem um sistema operacional próprio; em vez disso, compartilham diretamente o mesmo núcleo (Kernel) do Sistema Operacional da máquina hospedeira (host). O isolamento ocorre apenas nos processos e bibliotecas da aplicação, o que reduz seu tamanho para poucos megabytes e permite que iniciem de forma quase instantânea.

Em que situação você escolheria Serverless (Lambda) em vez de um container? E vice-versa?

Quando escolher Serverless (AWS Lambda): É ideal para tarefas de curta duração, execuções esporádicas ou arquiteturas totalmente baseadas em eventos (ex: processar um arquivo assim que ele é salvo no S3, ou rotinas de limpeza periódicas). A grande vantagem é que não há custos com servidores ociosos e a gerência de infraestrutura é zero, pagando-se estritamente pelos milissegundos de processamento utilizado.

When escolher Containers (Docker/ECS/EKS): É a escolha correta para aplicações de longa duração (long-running processes) que precisam ficar ativas 24/7 recebendo requisições contínuas, como websites tradicionais, APIs robustas e bancos de dados. Também é preferível quando há dependência de configurações muito específicas do sistema operacional ou quando se deseja evitar o aprisionamento tecnológico (vendor lock-in), visto que um container roda da mesma forma em qualquer nuvem ou infraestrutura local.